

A Novel Prime Numbers Based Hashing Technique for Minimizing Collisions

Rohit K. Bhullar

Chandigarh University
Punjab, INDIA
rohitbhullar@gmail.com

Lokesh Pawar

Chandigarh University
Punjab, INDIA,
lokesh.pawar@gmail.com

Vijay Kumar

Chandigarh University,
Punjab, INDIA,
nrw.vijaykumar@gmail.com

Anjali

DCSA, Panjab University,
Chandigarh, INDIA
anjdcsa@pu.ac.in

Abstract— Searching is a prime operation in computer science and numerous methods has been devised to make it efficient. Hashing is one such searching technique with objective of limiting the searching complexity to $O(1)$ i.e. finding the desired item in one attempt. But achieving complexity of $O(1)$ is quite difficult or usually not possible. This happens because there is no perfect mapping function for insertion and searching; and this imperfection of hashing function results in collisions. The algorithm and technique presented in this article minimizes the number of collisions by removing the problem of clustering. Clustering occurs when the data items congregates in one particular area thus increasing the number of collisions and results in increased number of probes to insert and search an item. During trials runs the proposed algorithm have shown considerable improvements over all major hashing algorithms in terms of performance.

Keywords: *Searching, hashing, collisions, search complexity, Algorithms*

I. INTRODUCTION

Hashing techniques have one obvious advantage of being best searching technique given no collision occurs [1]. But collisions occur and it's very difficult to stop them. The algorithm developed minimizes the number of collisions by removing the problem of clustering. This clustering causes uneven distribution of data items in storage area and results in wastage of resources i.e. memory space and makes insertion and retrieval inefficient [5].

Any hashing technique or has hash function can categorized as good if the output produced by the hash function is distributed uniformly across the hash table. The output of the hash function will be uniform across the hash table if, the input to the hash function is distributed or separated quantitatively in magnitude. In our technique we have achieved the separation among the input through the use of prime numbers [4]. Since the prime numbers have some sure difference among one another so, this fact can be utilized to make input uniformly distributed thus resolving the problem of collisions and clustering up to very large extent. During performance testing the proposed algorithm shows considerable improvements over other hashing techniques [10]. In this paper we are going to discuss about hashing technique which is a novel method for searching. This technique has several cases of design and formulations for ensuring less collision and efficient storage and searching. We have designed an approach using prime numbers with the storage address calculated by the value of hash-sum which is calculated by adding a value of prime number into the hash-sum. Doing this we will get a unique storage location and hence very less number of collisions.

Many hashing techniques and algorithms have been invented. Below is some brief introduction to some of these techniques and algorithms. The proposed method can be used in variety of applications including integrity check and for numbering nodes in networks. Different application of hashing can be seen in [13-19].

II. RELATED WORK

A. DIRECT HASHING TECHNIQUE (DHM): This technique is the simplest and straightforward and can be used if, the input data set is small. The input keys must

be unique [1-4]. In this technique the input key is mapped into the same array index (or the key itself act as an array index).

B. SUBTRACTION METHOD (SM): It is an improvisation over the direct hashing technique. This method is used if the key is larger than the array index i.e. magnitude[10]. Then the array index or address is calculated by subtracting some constant from the key. Rest of the functioning is same as that of the direct hashing technique.

C. MODULO DIVISION METHOD: Also known as division remainder, the modulo division method divides the key by the array size and uses the remainder for the address [11]. This method gives us the simple hashing algorithm and its formula is shown below. Here list size is the number of elements in the array:

$$\text{Address} = \text{key} \text{ MODULO list-size}$$

D. DIGIT EXTRACTION METHOD: Using digit extraction selected digits are extracted from the key and used as address [12]. For example, using six digit employee number to hash to a three digit address (000-999), we could select the first, third and fourth digits (from the left) and use them as address. See table 1 for working of this method.

TABLE 1: DIGIT EXTRACTION TABLE

Input	Output
379452	394
121267	112
378845	388
160252	102
045128	051

E. MID-SQUARE METHOD: In mid-square hashing the key is squared and the address is selected from the middle of the squared number. The most obvious limitation of this method is the size of the key. Given a key of six digits, the product will be 12 digits, which is beyond the maximum integer size of many computers. E.g. consider a key 9452, the mid square address

calculation is shown below using a four-digit address (0000-9999).

$$(9452)^2 = 89340304: \text{address is } 3403 \quad (i)$$

F. FOLDING METHODS: Two folding methods are used in literature and practice: fold shift and fold boundary.

In fold shift the key value is divided into parts whose size matches the size of the required address. Then the left and right parts are shifted and added with the middle part [6]. For example, imagine that we want to map social security number into three-digit addresses. We divide the nine-digit social security number into three-digit numbers, which are then added [10-12]. This method is shown in the following figure i.e. figure 1.

Fold Boundary: Assume the input key to be: 123456789. In fold boundary the left and right numbers are folded on a fixed boundary between

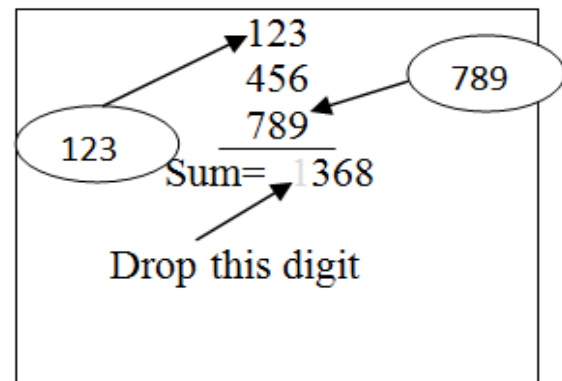


Fig. 1: Folding Method (Fold Shift)

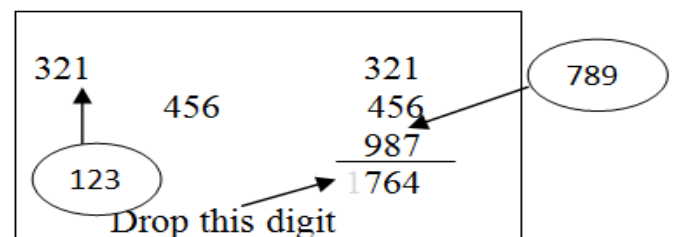


Fig. 2: Folding Method (Fold Boundary)

them and the center number. The two outer values are thus reversed, as shown in the figure 2, where 123 is folded to 321 and 789 is folded to 987. The middle part

is kept intact. Then all these three sets are added to form the hash key. While adding, drop the carry if any.

G. ROTATION METHOD: Rotation Hashing is generally not used by itself but, rather is incorporated in combination with other hashing methods [8]. It is most useful when keys are assigned serially, such as we often see in employee numbers and part numbers. Table 2 shown below explains the working mode if this method.

TABLE 2. ROTATION METHOD

Input key	Rotated key
600101	160010
600102	260010
600103	360010
600104	460010
600105	560010

H. PSEUDORANDOM HASHING: In pseudorandom hashing, the key is used as the seed in a pseudorandom number generator, and the resulting random number is then scaled into possible address range using modulo-division. Given a fixed seed, pseudorandom number generator always generates the same series of numbers[8]. This is what allows us to use them in hashing. E.g. consider a very common random-number generator below:

$$Y = ax + c \quad (ii)$$

To use the pseudorandom number generator as a hashing method, we set x to the key, multiply it by the coefficient a, then add the constant c. The result is then divided by the list size, with the remainder being the hashed address.

I. ASCII Based hashing algorithm: This technique is similar to direct hashing and in this algorithm the alphabets in the input strings are first converted into decimal number by the looking at the standard ASCII table[9].

III. PROPOSED RL HASHING ALGORITHM

In this technique all the alphabets in the character set have been assigned prime numbers (Like ASCII) which then have been used by the hash function to calculate the hashed address. E.g. the alphabet 'a' has been assigned a value of 1009 which is prime number alphabet 'b' has been assigned a value 1013 and c ,d ,e has been assigned values 1019,1021,1031 respectively (see table 3 for prime values of alphabets from a-z). In table 3 all the prime numbers have been assigned to alphabets in order of the occurrence of the prime number in series (This table is called PCII i.e. prime codes for information interchange). In this way a complete PCII table can be constructed for the alphabet set like an ASCII table but, for demonstration we have represented only 26 small case alphabets ranging between a-z including space (as space is also a character) [9]. The choice of prime numbers is discretionary but large prime numbers usually produces good separation among input and results in less numbers of collusions. In addition, the prime numbers should not be significantly large; otherwise during algorithm processing their sum can become greater than the limit of integer variables, which can reduce the efficiency. Consider a prime table (see table 3) for the 26 letter of English lowercase letters:

TABLE 3. ASSIGNED PRIME VALUES OF ALPHABETS FROM A-Z (PCII Table).

a=1009	b=1013	c=1019	d=1021	e=1031	f=1033
g=1039	h=1049	i=1051	j=1061	k=1063	l=1069
m=1087	n=1091	o=1093	p=1097	q=1103	r=1109
s=1117	t=1123	u=1129	v=1151	w=1153	x=1163
y=1175	z=1181	(space)= 1187			

Algorithm: RL Hash (key, LOC, PCII table, Maxsize, Item)

//Assumption: the **key** is a character string of length less than 16.

/* Array[Maxsize] is the hash table, hashedsum is a variable in which the sum of different character will be stored. */

```
/* Item is the data that we want
to search or insert and LOC is the
array index where the item is to
be inserted */
```

```
Set hashedsum to 0 ;
```

```
Set countchar to 0;
```

```
If (countchar < 16) /* here
16 is the maximum string
length */
```

```
{
```

```
For each character 'a' in
the key
```

```
{
```

```
Find out corresponding prime
number for a from the PCII
table ;
```

```
/* prime number will be
retrieved using table lookup
procedure */
```

```
Add prime number to
hashedsum;
```

```
i.e. hashedsum=hashedsum+
'a'
```

```
Increment countchar;
```

```
}
```

```
}
```

```
Else if (countchar > 16){
```

```
Input too large and Exit;
```

```
}
```

```
Else
```

```
{
```

```
LOC= hashedsum MODULO
(Maxsize + 1) ;
```

```
Array[LOC]=item; // if the
operation is insertion
```

```
Item=Array[loc]; // if the
operation is searching
```

```
}
```

```
End;
```

IV. CALCULATION OF HASH ADDRESS:

Suppose the input string is “dhairya” then its hashed address will be calculated as follows: We need to peep into prime table (table 3) to calculate the hash address [8-9]. Assume the list size=31 i.e. Maxsize is 31. We can see from the prime table that the codes for the alphabets for the given strings are:

d=1021, h=1049, a=1009, i=1051, r=1109, y=1175, a=1009.

After this step we need to sum up all these values which will be equal to: 7423. This sum will be stored in the variable **hashed-sum**. After this we will apply the hash function that is a simple modulo division i.e.

Hash-address = (hashed-sum % size of hash table+1)

Consider one more example where the input string is “rohit” looking at table 3 we get the following values for the alphabets of the string: r=1109, o=1093, h=1049, i=1051, t=1123. Now the hashed-sum will be = 5425, then after applying the hash function we get the hash address of 17, i.e the hash-address will be 17, so the name rohit will be stored at location 17.

Here hash-address will be the final storage location where the input string will be stored. From the above calculations this comes out to be 31, so this name will be stored at an array index of 31.

Similarly other items can be inserted in the array or hash table. The retrieval operation will work in similar order.

Array Index	String
001	
002	
003	
004	
005	
006	
007	
008	
009	
010	
011	
012	
013	
014	
015	
016	
017	rohit
018	
019	
020	
021	
022	
023	
•	
•	
•	
•	
031	dhairya

Fig 3. Hash Table with Sample Data.

In this way can calculate the value of all the strings. Figure 3 demonstrates this insertion. Size of the hash table is the total size of the hash table. The array size chosen should be a prime number and should be larger than the total number of elements that we want to store for efficiency considerations. The load factor of the hash table should be less than 70 percent of the total number of elements.

V. RESULTS AND DISCUSSION

We have performed extensive executions in C language for measuring the performance of the proposed RL algorithm and compared its output with existing benchmark hashing algorithms. The comparison has

been based on the number of collision incurred by different algorithms keeping the rest of the environment similar.

Assumptions made: Array or Linked List Size: 20,31,40,50 for different iterations has been taken for performance evaluation.

The algorithms used for comparison are direct hashing, pseudorandom hashing and RL algorithm. The number of collision for a different given memory space has been observed. Form figure 4 it is clear that the proposed algorithm outperforms other algorithm. The blue, Red, Green lines indicates the direct, pseudorandom and RL hashing Algorithm.

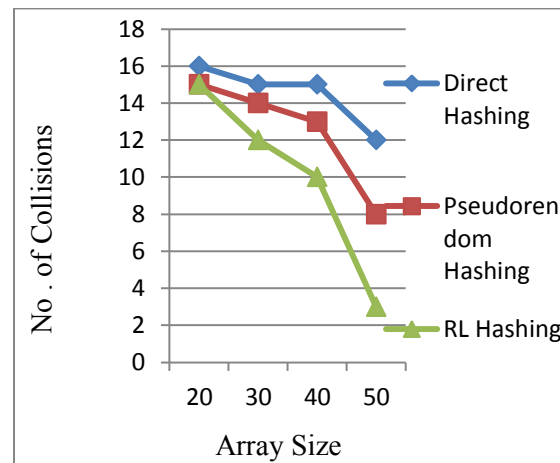


Fig 4: Collision Comparison of Different Algorithms

The proposed algorithm has been compared with other algorithms also. These algorithms are modulo division and mid-square algorithms. The number of collision for a different given memory space has been observed similarly as done before. Form figure 5 it is clear that the proposed algorithm outperforms other algorithm. The blue, Red, Green lines indicates the direct, pseudorandom and RL hashing Algorithm.

VI. CONCLUSION

Hashing techniques have one obvious advantage of being best searching technique given no collision occurs. But collisions occur and it's very difficult to stop them. The algorithm developed minimizes the number of collisions by removing the problem of clustering.

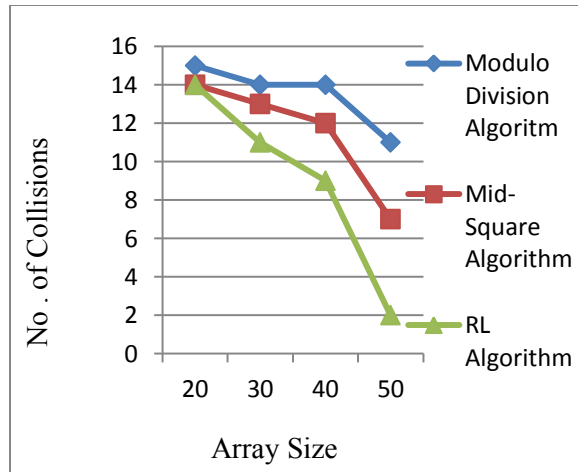


Fig. 5: Collision Comparison.

This clustering degrades the system performance and results in inefficient resource utilization. The wide gap in input domain supported by prime numbers plays a key role in efficiency of this approach. The input separation makes a strong foundation for robust and efficient hashing function. While evaluation, the proposed algorithm have surpassed all major hashing algorithms in terms of performance.

The abilities of prime numbers have been exploited to make this scheme successful. The proposed method can also be used for integrity testing of data and can be used for enforcing security in network communications.

REFERENCES:

- [1] X. Shen et al., Multi-view Latent Hashing for Efficient Multimedia Search, Proceedings MM'15, Pg No. 831-834, 2015.
- [2] G.Lin et al., Fast Supervised Hashing with Decision Trees for High-Dimensional Data, IEEE Conference on Computer Vision and Pattern Recognition, Pg. No. 1971-1978, 2014.
- [3] E. Andreeva et al., A three property secure hash function, SAC, Pg. No. 228-244, 2008.
- [4] J. Philippe et al., The Hash Function Family LAKE, IACR, Pg. No. 36-53, 2008.
- [5] D. Belazzougui et al., Hash, Displace and compress, NISTW, 2008.
- [6] S. Lefebvre et al., Perfect Spatial Hashing, Proceedings SIGGRAPH'06 ACM, PG No. 579-588, 2006.
- [7] G. Kollios et al., Hashing Methods for temporal data, IEEE Transactions on Knowledge and Data Engineering, Volume 14, Pg No. 902-919, 2002.
- [8] A. Gionis et al., Similarity search in high dimension via Hashing, Proceedings of the 25th VLDB Conference Edinburgh, Scotland, 1999.
- [9] H. Wolfson et al., Model-based object recognition by geometric hashing, ECCV Springer, 1990.
- [10] M. Ramakrishna et al., Hashing Practice: Analysis of Hashing and Universal Hashing, Proceedings Sigmond, Pg. No. 191-199, 1988.
- [11] P. Larson, Dynamic Hashing, BIT Numerical Mathematics, Pg. No. 184-201, 1978.
- [12] J.C. Grey et al., Compound Data Structures for computer aided design, a survey ACM 67 Proceedings of the 1967.
- [13] K. Taneja et al., "EEGRP: Grid Based and Caching Equipped Energy Efficient Routing Protocol for MANET with Restricted Movement", Far East Journal of Electronics and Communications, ISSN: 0973-6999, pp- 1-16, (2016).
- [14] Taneja, Kavita, Harmunish Taneja, and Rohit Kumar. "SPF: Segmented processor framework for energy efficient proactive routing based applications in MANET." 2015 2nd International Conference on Recent Advances in Engineering & Computational Sciences (RAECS). IEEE, 2015.
- [15] Rohit Kumar et al., "Specialized hardware Architecture for Smartphones", IJERA, ISSN: 2248-9622 Volume 4, Issue 5, 2014.
- [16] L. pawar et al., "Optimized Route Selection On The Basis Of Discontinuity And Energy Consumption In Delay Tolerant Networks", Springer : Advances in Intelligent Systems and Computing, ISSN: 2194-5357, In Press.
- [17] Rohit K. Bhullar et al., "Novel Stress Calculation in Segmented Processor Systems Using Buddy Approach", Journal of intelligent and fuzzy systems" IOS press, ISSN: 1064-1246, In Press.
- [18] G. Bathla et al. "Prolonging the network lifetime of Tier Based Minimum Spanning Tree Structured wireless sensor Networks", International Journal on Cloud Computing: Services and Architecture (IJCCSA), Vol. 1, No. 1, pp. 12-20, May 2011.
- [19] Taneja, Kavita, Harmunish Taneja, and Rohit K. Bhullar, "Cross-Platform Application Development for Smartphones: Approaches and Implications", Proceedings of the 10th INDIACom-2016; IEEE Conference ID: 37465 2016, 16th - 18th March, 2016